

3장 4지선다형 60문항

반복문

1. 자바에서 제공하는 반복문의 종류가 아닌 것은?

- ① `for` 문
- ② `while` 문
- ③ `do-while` 문
- ④ `repeat-until` 문

2. 반복 횟수가 명확한 경우 일반적으로 가장 적합한 반복문은?

- ① `for` 문
- ② `while` 문
- ③ `switch` 문
- ④ `if` 문

3. `for` 문의 기본 구성 순서로 올바른 것은?

- ① 조건식, 초기식, 증감식
- ② 초기식, 조건식, 증감식
- ③ 증감식, 조건식, 초기식
- ④ 초기식, 증감식, 조건식

4. 다음 코드에서 반복되는 횟수는?

```
for (int i = 0; i < 10; i++) {  
    System.out.print(i);  
}
```

- ① 9회
- ② 10회
- ③ 11회
- ④ 무한 반복

5. 다음 코드의 출력 결과는?

```
for (int i = 0; i < 5; i++) {  
    System.out.print(i + " ");  
}
```

- ① 0 1 2 3 4

② 0 1 2 3 4 5

③ 1 2 3 4 5

④ 1 2 3 4

6. 다음 코드의 출력 결과는?

```
for (int i = 1; i <= 5; i++) {  
    System.out.print(i + " ");  
}
```

① 0 1 2 3 4

② 1 2 3 4

③ 1 2 3 4 5

④ 0 1 2 3 4 5

7. 다음 코드 실행 후 **sum** 의 값은?

```
int sum = 0;  
  
for (int i = 1; i <= 5; i++) {  
    sum += i;  
}
```

① 5

② 10

③ 15

④ 20

8. 다음 중 무한 반복을 만드는 **for** 문은?

① for (int i = 0; i < 10; i++)

② for (int i = 10; i > 0; i--)

③ for (;;)

④ for (int i = 0; i == 10; i++)

9. 다음 코드에서 변수 **i** 를 사용할 수 있는 범위는?

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```

① 프로그램 전체

② 클래스 전체

③ `main()` 메소드 전체

④ `for` 문 내부

10. 다음 코드의 실행 결과는?

```
for (int i = 3; i >= 1; i--) {  
    System.out.print(i + " ");  
}
```

① 1 2 3

② 3 2 1

③ 3 2 1 0

④ 2 1 0

11. `while` 문에 대한 설명으로 올바른 것은?

① 작업을 실행한 후 조건을 검사한다.

② 조건이 `true` 인 동안 작업을 반복한다.

③ 조건 없이 사용할 수 있다.

④ 반드시 한 번 이상 작업을 실행한다.

12. 다음 코드의 반복 횟수는?

```
int i = 0;  
  
while (i < 3) {  
    i++;  
}
```

① 0회

② 2회

③ 3회

④ 4회

13. 다음 코드의 출력 결과는?

```
int i = 1;  
  
while (i <= 3) {  
    System.out.print(i + " ");  
    i++;  
}
```

① 0 1 2

② 1 2 3

③ 1 2 3 4

④ 3 2 1

14. **while** 문의 조건식이 처음부터 **false** 인 경우는?

- ① 작업을 한 번 실행한다.
- ② 작업을 두 번 실행한다.
- ③ 작업을 실행하지 않는다.
- ④ 무한 반복한다.

15. 다음 중 **while** 문의 무한 반복을 나타내는 코드는?

① `while (false)`

② `while (true)`

③ `while (0)`

④ `while ()`

16. 다음 코드에서 **1**의 역할은?

```
while (n != -1) {  
    sum += n;  
    n = scanner.nextInt();  
}
```

- ① 합계에 반드시 포함하는 값
- ② 반복을 종료하는 입력값
- ③ 반복 횟수를 저장하는 값
- ④ 평균을 저장하는 값

17. **do-while** 문에 대한 설명으로 올바른 것은?

- ① 조건을 먼저 검사한다.
- ② 작업을 최소 한 번 실행한다.
- ③ 조건식을 작성하지 않는다.
- ④ 무한 반복을 만들 수 없다.

18. 다음 코드의 출력 결과는?

```
int i = 5;  
  
do {  
    System.out.print(i);
```

```
i++;
} while (i < 5);
```

- ① 아무것도 출력되지 않는다.
- ② 4
- ③ 5
- ④ 무한 반복한다.

19. do-while 문의 문법으로 올바른 것은?

①

```
do (조건식) {
    작업문;
}
```

②

```
do {
    작업문;
} while (조건식);
```

③

```
do {
    작업문;
} while;
```

④

```
do while (조건식) {
    작업문;
}
```

20. 다음 코드가 출력하는 문자의 범위는?

```
char c = 'a';

do {
    System.out.print(c);
    c = (char)(c + 1);
} while (c <= 'z');
```

- ① a 부터 y 까지
- ② a 부터 z 까지

③ b 부터 z 까지

④ A 부터 Z 까지

중첩 반복문과 분기문

21. 중첩 반복문에 대한 설명으로 올바른 것은?

- ① 반복문 안에 다른 반복문을 작성한 구조이다.
- ② 반복문 다음에 조건문을 작성한 구조이다.
- ③ 두 프로그램을 동시에 실행하는 구조이다.
- ④ 한 반복문을 두 번 호출하는 구조이다.

22. 다음 코드에서 `System.out.print("*");` 의 실행 횟수는?

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 2; j++) {  
        System.out.print("*");  
    }  
}
```

- ① 2회
- ② 3회
- ③ 5회
- ④ 6회

23. 다음 코드의 출력 결과는?

```
for (int i = 1; i <= 2; i++) {  
    for (int j = 1; j <= 3; j++) {  
        System.out.print(i);  
    }  
}
```

- ① 111222
- ② 123123
- ③ 112233
- ④ 121212

24. 구구단 전체를 출력하는 중첩 반복문에서 바깥쪽 반복문의 일반적인 역할은?

- ① 곱하는 수를 1부터 9까지 변경한다.
- ② 단을 1단부터 9단까지 변경한다.
- ③ 프로그램을 종료한다.

- ④ 출력 결과를 문자열로 변환한다.

25. 중첩 반복문 내부에서 일반적인 **break** 문을 실행하면 어떻게 되는가?

- ① 프로그램 전체가 종료된다.
- ② 모든 반복문을 한 번에 벗어난다.
- ③ 가장 안쪽 반복문 하나를 벗어난다.
- ④ 바깥쪽 반복문만 벗어난다.

26. **break** 문의 기능으로 올바른 것은?

- ① 현재 반복문을 벗어난다.
- ② 현재 반복만 건너뛰는다.
- ③ 반복문의 처음으로 이동한다.
- ④ 변수의 값을 초기화한다.

27. 다음 코드의 출력 결과는?

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        break;  
    }  
    System.out.print(i + " ");  
}
```

- ① 1 2
- ② 1 2 3
- ③ 1 2 4 5
- ④ 3 4 5

28. **continue** 문의 기능으로 올바른 것은?

- ① 반복문을 완전히 종료한다.
- ② 프로그램을 종료한다.
- ③ 현재 반복의 나머지 부분을 건너뛰고 다음 반복으로 진행한다.
- ④ 조건식을 삭제한다.

29. 다음 코드의 출력 결과는?

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
}
```

```
System.out.print(i + " ");  
}
```

- ① 1 2
- ② 1 2 3 4 5
- ③ 1 2 4 5
- ④ 3

30. 다음 코드 실행 후 **sum** 의 값은?

```
int sum = 0;  
  
for (int i = 1; i <= 5; i++) {  
    if (i % 2 == 0) {  
        continue;  
    }  
    sum += i;  
}
```

- ① 6
- ② 9
- ③ 12
- ④ 15

1차원 배열

31. 배열에 대한 설명으로 가장 적절한 것은?

- ① 서로 다른 자료형의 값을 무조건 저장하는 구조이다.
- ② 같은 자료형의 여러 값을 하나의 이름으로 관리하는 구조이다.
- ③ 하나의 값만 저장할 수 있는 변수이다.
- ④ 반복문을 대신하여 사용하는 명령문이다.

32. 정수 5개를 저장할 배열을 올바르게 생성한 것은?

- ① `int scores = new int[5];`
- ② `int[] scores = new int[5];`
- ③ `int[5] scores = new int;`
- ④ `array scores = int[5];`

33. 다음 중 자바의 올바른 배열 선언은?

- ① `int[] numbers;`
- ② `int numbers[5];`

③ `array int numbers;`

④ `numbers int[];`

34. 다음 배열에서 첫 번째 원소의 인덱스는?

```
int[] numbers = {10, 20, 30};
```

① 0

② 1

③ 2

④ 3

35. 다음 코드의 실행 결과는?

```
int[] numbers = {10, 20, 30};  
System.out.println(numbers[1]);
```

① 10

② 20

③ 30

④ 오류

36. 다음 배열에서 마지막 원소에 접근하는 식은?

```
int[] numbers = {10, 20, 30, 40};
```

① `numbers[4]`

② `numbers[3]`

③ `numbers[length]`

④ `numbers[-1]`

37. 다음 코드의 실행 결과는?

```
int[] numbers = {10, 20, 30, 40};  
System.out.println(numbers.length);
```

① 3

② 4

③ 5

④ 40

38. 다음 코드에서 오류가 발생하는 부분은?

```
int[] numbers = new int[3];
numbers[0] = 10;
numbers[1] = 20;
numbers[3] = 30;
```

① 배열 선언 부분

② `numbers[0] = 10`

③ `numbers[1] = 20`

④ `numbers[3] = 30`

39. `new int[5]` 로 배열을 생성했을 때 각 원소의 기본값은?

① `-1`

② `0`

③ `null`

④ `1`

40. 다음 코드 실행 후 `sum` 의 값은?

```
int[] numbers = {1, 2, 3, 4};
int sum = 0;

for (int i = 0; i < numbers.length; i++) {
    sum += numbers[i];
}
```

① 4

② 6

③ 10

④ 24

41. 배열의 모든 원소를 안전하게 반복하기 위한 조건식은?

① `i <= array.length`

② `i < array.length`

③ `i == array.length`

④ `i > array.length`

42. 다음 `for-each` 문에서 변수 `n` 에 차례로 저장되는 것은?

```
int[] numbers = {10, 20, 30};

for (int n : numbers) {
```

```
System.out.println(n);  
}
```

- ① 배열의 인덱스
- ② 배열의 원소 값
- ③ 배열의 길이
- ④ 배열의 주소만

43. 다음 코드의 실행 결과는?

```
int[] numbers = {1, 2, 3};  
int sum = 0;  
  
for (int n : numbers) {  
    sum += n;  
}  
  
System.out.println(sum);
```

- ① 3
- ② 5
- ③ 6
- ④ 9

44. **for-each** 문이 유용한 경우는?

- ① 배열의 모든 원소를 순차적으로 읽을 때
- ② 특정 인덱스를 직접 변경해야 할 때만
- ③ 배열의 크기를 변경할 때
- ④ 새로운 자료형을 선언할 때

2차원 배열

45. 정수형 2차원 배열의 올바른 선언은?

- ① `int[] numbers[];`
- ② `int[][] numbers;`
- ③ ①과 ② 모두 가능하다.
- ④ ①과 ② 모두 사용할 수 없다.

46. 다음 코드로 생성되는 배열의 행과 열은?

```
int[][] numbers = new int[2][5];
```

- ① 2행 5열
- ② 5행 2열
- ③ 2행 2열
- ④ 5행 5열

47. 다음 2차원 배열에서 `numbers.length` 의 값은?

```
int[][] numbers = new int[2][5];
```

- ① 2
- ② 5
- ③ 7
- ④ 10

48. 다음 2차원 배열에서 `numbers[0].length` 의 값은?

```
int[][] numbers = new int[2][5];
```

- ① 0
- ② 2
- ③ 5
- ④ 10

49. 다음 코드의 실행 결과는?

```
int[][] numbers = {  
    {1, 2, 3},  
    {4, 5, 6}  
};  
  
System.out.println(numbers[1][2]);
```

- ① 3
- ② 4
- ③ 5
- ④ 6

50. 다음 배열에서 행의 개수와 첫 번째 행의 열 개수를 올바르게 나타낸 것은?

```
int[][] data = {  
    {1, 2},  
    {3, 4},  
};
```

```
{5, 6}  
};
```

- ① 2행 3열
- ② 3행 2열
- ③ 3행 3열
- ④ 6행 1열

배열과 메소드

51. 배열을 메소드의 매개변수로 전달하는 선언으로 올바른 것은?

- ① `void printArray(int[] array)`
- ② `void printArray(int array())`
- ③ `void printArray(array int)`
- ④ `void printArray(int[5] array)`

52. 다음 메소드 호출 후 배열의 첫 번째 원소 값은?

```
static void change(int[] array) {  
    array[0] = 100;  
}  
  
public static void main(String[] args) {  
    int[] numbers = {10, 20};  
    change(numbers);  
}
```

- ① 0
- ② 10
- ③ 20
- ④ 100

53. 배열을 반환하는 메소드의 선언으로 올바른 것은?

- ① `static int makeArray()`
- ② `static int[] makeArray()`
- ③ `static array makeArray()`
- ④ `static void[] makeArray()`

54. 다음 메소드에서 반환해야 하는 값으로 적절한 것은?

```
static int[] makeArray() {  
    int[] temp = new int[4];  
    // 반환문  
}
```

- ① `return 4;`
- ② `return int;`
- ③ `return temp;`
- ④ `return temp.length;`

main() 메소드와 명령행 인자

55. 자바 응용프로그램의 실행 시작점은?

- ① `start()`
- ② `run()`
- ③ `main()`
- ④ `execute()`

56. `main()` 메소드의 올바른 선언은?

- ① `public void main(String args)`
- ② `public static void main(String[] args)`
- ③ `static int main(String[] args)`
- ④ `private static void main(int[] args)`

57. `main()` 메소드의 반환형은?

- ① `int`
- ② `String`
- ③ `boolean`
- ④ `void`

58. `main()` 메소드에서 명령행 인자를 저장하는 매개변수의 자료형은?

- ① `int[]`
- ② `char[]`
- ③ `String[]`
- ④ `double[]`

59. 명령행 인자의 개수를 확인하는 코드는?

- ① `args.size`
- ② `args.length`

③ `args.count()`

④ `args.size()`

60. 문자열 "20.5" 를 double 값으로 변환하는 코드는?

① `Integer.parseInt("20.5")`

② `String.parseDouble("20.5")`

③ `Double.parseDouble("20.5")`

④ `double("20.5")`

정답표

문항	정답	문항	정답	문항	정답
1	④	21	①	41	②
2	①	22	④	42	②
3	②	23	①	43	③
4	②	24	②	44	①
5	①	25	③	45	③
6	③	26	①	46	①
7	③	27	①	47	①
8	③	28	③	48	③
9	④	29	③	49	④
10	②	30	②	50	②
11	②	31	②	51	①
12	③	32	②	52	④
13	②	33	①	53	②
14	③	34	①	54	③
15	②	35	②	55	③
16	②	36	②	56	②
17	②	37	②	57	④
18	③	38	④	58	③
19	②	39	②	59	②
20	②	40	③	60	③

3장 4지선다형 60문항 해설

1~10. for 문

문항	정답	해설
1	④	자바의 반복문에는 <code>for</code> , <code>while</code> , <code>do-while</code> 이 있다. <code>repeat-until</code> 은 자바에서 제공하는 반복문이 아니다.
2	①	반복 횟수가 명확한 경우에는 초기식·조건식·증감식을 한 곳에서 관리할 수 있는 <code>for</code> 문이 일반적으로 적합하다.
3	②	<code>for</code> 문의 기본 구조는 <code>for(초기식; 조건식; 증감식)</code> 순서이다.
4	②	<code>i</code> 는 0부터 9까지 변하므로 반복문은 총 10회 실행된다.
5	①	<code>i=0</code> 부터 <code>i<5</code> 인 동안 출력하므로 0 1 2 3 4 가 출력된다.
6	③	<code>i=1</code> 부터 <code>i<=5</code> 까지 실행하므로 1 2 3 4 5 가 출력된다.
7	③	<code>sum</code> 에 1부터 5까지 더하므로 1+2+3+4+5=15 이다.
8	③	<code>for(;;)</code> 는 초기식, 조건식, 증감식을 모두 생략한 형태로 조건 없이 계속 반복한다.
9	④	<code>for</code> 문의 초기식에서 선언한 <code>int i</code> 는 해당 <code>for</code> 문 내부에서 사용할 수 있다.
10	②	<code>i=3</code> 에서 시작하여 매번 1씩 감소하고 <code>i>=1</code> 까지 반복하므로 3 2 1 이 출력된다.

1~10번은 `for` 문의 구조, 반복 횟수, 변수 범위와 코드 실행 결과를 확인하는 문제이다.

11~20. while 문과 do-while 문

문항	정답	해설
11	②	<code>while</code> 문은 조건식이 <code>true</code> 인 동안 작업문을 반복한다.
12	③	<code>i</code> 가 0, 1, 2일 때 실행되고 각각 1씩 증가하므로 총 3회 반복한다.
13	②	<code>i=1</code> 부터 시작하여 <code>i<=3</code> 인 동안 출력하므로 1 2 3 이 출력된다.
14	③	<code>while</code> 문은 작업문 실행 전에 조건을 검사하므로 처음부터 조건이 <code>false</code> 이면 한 번도 실행하지 않는다.
15	②	<code>while(true)</code> 는 조건이 항상 참이므로 별도의 종료 조건이 없다면 무한 반복한다.
16	②	코드의 종료값은 <code>-1</code> 이다. <code>n</code> 에 -1이 입력되면 <code>n != -1</code> 이 거짓이 되어 반복이 종료된다.
17	②	<code>do-while</code> 은 작업문을 먼저 실행한 뒤 조건을 검사하므로 작업문이 최소 한 번 실행된다.
18	③	처음 <code>i=5</code> 이고 조건 검사 전에 <code>System.out.print(i)</code> 가 실행되므로 5 가 한 번 출력된다.
19	②	올바른 형식은 <code>do { 작업문; } while(조건식);</code> 이다. 끝에 세미콜론이 필요하다.
20	②	<code>'a'</code> 부터 출력한 뒤 문자를 하나씩 증가시키고 <code>'z'</code> 까지 조건을 만족하므로 a 부터 z 까지 출력한다.

11~20번은 `while` 과 `do-while` 의 조건 검사 시점 및 실행 결과를 다룬다.

21~30. 중첩 반복문과 break·continue

문항	정답	해설
21	①	반복문 안에 또 다른 반복문을 작성한 구조를 중첩 반복문 이라고 한다.
22	④	바깥쪽 반복문이 3회, 안쪽 반복문이 각 2회 실행되므로 $3 \times 2 = 6$ 회 출력된다.
23	①	<code>i=1</code> 일 때 1을 세 번 출력하고 <code>i=2</code> 일 때 2를 세 번 출력하므로 111222 이다.
24	②	구구단에서는 일반적으로 바깥쪽 반복문이 단을 변경하고 안쪽 반복문이 곱하는 수를 변경한다.
25	③	일반적인 <code>break</code> 는 자신이 포함된 가장 안쪽 반복문 하나만 종료한다.
26	①	<code>break</code> 는 현재 실행 중인 반복문을 즉시 벗어난다.
27	①	<code>i=1</code> , 2 는 출력되고 <code>i=3</code> 에서 <code>break</code> 가 실행되어 반복문이 종료된다. 따라서 12 가 출력된다.
28	③	<code>continue</code> 는 반복문 전체를 종료하지 않고 현재 반복의 나머지 부분을 건너뛰어 다음 반복으로 진행한다.
29	③	<code>i=3</code> 일 때 <code>continue</code> 가 실행되어 3만 출력하지 않는다. 따라서 1 2 4 5 가 출력된다.
30	②	짝수 2와 4는 <code>continue</code> 로 제외된다. 따라서 $1+3+5=9$ 가 <code>sum</code> 에 저장된다.

21~30번은 중첩 반복문의 실행 횟수와 `break` , `continue` 의 동작을 확인한다.

31~44. 1차원 배열

문항	정답	해설
31	②	배열은 같은 자료형의 여러 값을 하나의 이름으로 관리하는 자료구조 이다.
32	②	정수 5개를 저장할 배열은 <code>int[] scores = new int[5];</code> 로 생성한다.
33	①	<code>int[] numbers;</code> 는 올바른 자바 배열 선언이다.
34	①	배열의 인덱스는 0부터 시작 한다. 따라서 첫 번째 원소의 인덱스는 0이다.
35	②	<code>numbers[1]</code> 은 두 번째 원소를 의미하므로 값은 20 이다.
36	②	원소가 4개인 배열의 인덱스는 0~3이므로 마지막 원소는 <code>numbers[3]</code> 이다.
37	②	<code>numbers.length</code> 는 배열에 저장할 수 있는 원소의 개수이므로 4 이다.
38	④	길이가 3인 배열의 유효한 인덱스는 0, 1, 2이다. <code>numbers[3]</code> 은 범위를 벗어나므로 오류가 발생한다.
39	②	<code>new int[5]</code> 로 생성된 정수 배열의 각 원소는 기본적으로 0 으로 초기화된다.
40	③	배열의 모든 값을 더하면 $1+2+3+4=10$ 이므로 <code>sum</code> 은 10 이다.
41	②	배열 인덱스는 0부터 <code>length-1</code> 까지이므로 반복 조건은 <code>i < array.length</code> 가 안전하다.
42	②	향상된 <code>for</code> 문에서 <code>n</code> 에는 인덱스가 아니라 배열의 원소 값 이 차례로 저장된다.
43	③	<code>n</code> 에 1, 2, 3이 차례로 저장되고 모두 더하므로 결과는 6 이다.
44	①	<code>for-each</code> 문은 배열의 모든 원소를 처음부터 끝까지 순차적으로 읽을 때 유용하다.

31~44번은 배열 선언과 생성, 인덱스, `length` , 기본값 및 `for-each` 문을 다룬다.

45~50. 2차원 배열

문항	정답	해설
45	③	자바에서는 <code>int[] numbers[]</code> ; 와 <code>int[][] numbers</code> ; 모두 2차원 배열 선언으로 사용할 수 있다.
46	①	<code>new int[2][5]</code> 는 2행 5열의 2차원 배열을 생성한다.
47	①	2차원 배열에서 <code>numbers.length</code> 는 행의 개수이다. 따라서 2이다.
48	③	<code>numbers[0].length</code> 는 첫 번째 행의 열 개수이므로 5이다.
49	④	<code>numbers[1][2]</code> 는 두 번째 행의 세 번째 값이다. <code>{4,5,6}</code> 의 세 번째 값이므로 6이다.
50	②	배열에는 <code>{1,2}</code> , <code>{3,4}</code> , <code>{5,6}</code> 의 세 행이 있고 각 행에 두 개의 값이 있으므로 3행 2열이다.

2차원 배열에서는 첫 번째 인덱스가 행, 두 번째 인덱스가 열을 나타낸다.

51~54. 배열과 메소드

문항	정답	해설
51	①	배열을 매개변수로 받을 때는 <code>void printArray(int[] array)</code> 와 같이 배열 자료형을 지정한다.
52	④	<code>change(numbers)</code> 를 호출하면 <code>array</code> 가 같은 배열을 가리킨다. <code>array[0]=100</code> 이 실행되므로 <code>numbers[0]</code> 도 100이 된다.
53	②	<code>int[]</code> 배열을 반환하는 메소드는 <code>static int[] makeArray()</code> 와 같이 반환 자료형을 <code>int[]</code> 로 선언한다.
54	③	메소드의 반환 자료형이 <code>int[]</code> 이므로 생성한 배열을 가리키는 <code>temp</code> 를 <code>return temp;</code> 로 반환해야 한다.

51~54번은 배열을 메소드의 매개변수로 전달하거나 배열 자체를 반환하는 방법을 확인한다.

55~60. main() 메소드와 명령행 인자

문항	정답	해설
55	③	자바 응용프로그램의 실행 시작점은 <code>main()</code> 메소드이다.
56	②	올바른 선언은 <code>public static void main(String[] args)</code> 이다.
57	④	<code>main()</code> 은 값을 반환하지 않으므로 반환형은 <code>void</code> 이다.
58	③	명령행 인자는 문자열의 배열로 전달되므로 매개변수의 자료형은 <code>String[]</code> 이다.
59	②	<code>args</code> 는 배열이므로 명령행 인자의 개수는 <code>args.length</code> 로 확인한다.
60	③	문자열 <code>"20.5"</code> 를 <code>double</code> 값으로 변환하려면 <code>Double.parseDouble("20.5")</code> 을 사용한다.

55~60번은 `main()` 메소드의 선언 형식과 `String[] args` , 명령행 인자 및 문자열의 실수 변환을 다룬다.